4 Queens using Hill Climbing Algorithm

Code:

```
import random
import math

N = 8
INITIAL_TEMPERATURE = 10.0
COOLING_RATE = 0.995
MIN_TEMPERATURE = 0.01
MAX_ITERATIONS = 5000

def cost(board):
    """Count all attacking pairs of queens."""
    conflicts = 0

    for i in range(N):
        for j in range(i + 1, N):
            same_row = board[i] == board[j]
            same_diagonal = (
                abs(board[i] - board[j]) == abs(i - j)
            )

            if same_row or same_diagonal:
                conflicts += 1

    return conflicts


def generate_neighbor(board):
    """Move one randomly chosen queen to a new row."""
    neighbor = board.copy()
    col = random.randrange(N)
    old_row = neighbor[col]
    possible_rows = [r for r in range(1, N + 1)
                     if r != old_row]
    neighbor[col] = random.choice(possible_rows)
    return neighbor


def print_grid(board):
    """Print the N-Queens solution as an N x N grid."""
    print("\nGrid Representation:")
    for row in range(1, N + 1):
        line = ["Q" if board[col] == row else "." for col in range(N)]
        print(" ".join(line))

def simulated_annealing():
    restart = 0

    while True:
        restart += 1
        current = [
```

```
            random.randint(1, N) for _ in range(N)
        ]
        current_cost = cost(current)
        temperature = INITIAL_TEMPERATURE

        print(f"\n========== Restart {restart} ==========")
        print("Initial state:", current)
        print("Initial cost:", current_cost)

        for step in range(1, MAX_ITERATIONS + 1):

            if current_cost == 0:
                print("\nGOAL FOUND!")
                print("Goal configuration:", current)
                print("Final cost:", current_cost)
                print("Steps in this restart:", step - 1)
                print_grid(current)
                return current

            if temperature < MIN_TEMPERATURE:
                break

            neighbor = generate_neighbor(current)
            neighbor_cost = cost(neighbor)

            delta = neighbor_cost - current_cost
            if delta <= 0:
                probability = 1.0
                accept = True
            else:
                probability = math.exp(
                    -delta / temperature)
                accept = random.random() < probability

            print(f"\nStep {step}")
            print("Current state:", current)
            print("Current cost:", current_cost)
            print("Neighbor state:", neighbor)
            print("Neighbor cost:", neighbor_cost)
            print("Delta E:", delta)
            print(f"Temperature: {temperature:.4f}")
            print(f"Acceptance probability: {probability:.4f}")
            print("Decision:", "ACCEPT" if accept else "REJECT")

            if accept:
                current = neighbor
                current_cost = neighbor_cost

            temperature *= COOLING_RATE

        print("No solution in this run. Restarting...")

if __name__ == "__main__":
    simulated_annealing()
```

Output:

```
Step 852
Current state: [3, 6, 8, 1, 4, 7, 5, 8]
Current cost: 1
Neighbor state: [3, 6, 8, 1, 5, 7, 5, 8]
Neighbor cost: 3
Delta E: 2
Temperature: 0.1404
Acceptance probability: 0.0000
Decision: REJECT

Step 853
Current state: [3, 6, 8, 1, 4, 7, 5, 8]
Current cost: 1
Neighbor state: [3, 6, 8, 1, 4, 7, 5, 7]
Neighbor cost: 2
Delta E: 1
Temperature: 0.1397
Acceptance probability: 0.0008
Decision: REJECT

Step 854
Current state: [3, 6, 8, 1, 4, 7, 5, 8]
Current cost: 1
Neighbor state: [3, 6, 8, 1, 4, 7, 5, 2]
Neighbor cost: 0
Delta E: -1
Temperature: 0.1390
Acceptance probability: 1.0000
Decision: ACCEPT

GOAL FOUND!
Goal configuration: [3, 6, 8, 1, 4, 7, 5, 2]
Final cost: 0
Steps in this restart: 854

Grid Representation:
. . . Q . . . .
. . . . . . . Q
Q . . . . . . .
. . . . Q . . .
. . . . . . Q .
. Q . . . . . .
. . . . . Q . .
. . Q . . . . .
PS C:\Users\Administrator\Desktop\1BM24CS316>
```